ICB-MergFS: A Request-Merging ICB-to-AHB-Lite Bridge with Assertion-Based Verification for RISC-V SoCs

Dr. Atul Kumar Nishad
Assistant Professor
Department of Electronics and Communication Engineering
National Institute of Technology Warangal,India

Priyanshi Agarwal
Department of Electronics and Communication Engineering
National Institute of Technology Warangal, India
pa23ecb1a19@student.nitw.ac.in

***Abstract*—Modern System-on-Chip (SoC) architectures consist of multiple bus domains operating at different clock frequencies, data widths, and communication protocols. Efficient communication between these domains is essential to achieve high system performance while maintaining low latency and resource utilization. Conventional bus bridges that connect a 32-bit intra-core bus to a 64-bit AHB-Lite bus often process every transaction independently. As a result, only half of the available data bandwidth of the wider bus is utilized, and additional latency is introduced during communication across different clock domains.**

This work presents ICB-MergFS, a bridge that adapts the request-merging and frequency-aware buffering principles of the MergFS architecture [1], originally designed for a proprietary CBus protocol to the open Intra-Core Bus (ICB) protocol used in the Hummingbird E203 RISC-V core. It connects a 32-bit Internal Chip Bus (ICB) with a 64-bit AHB-Lite bus. The bridge identifies eligible adjacent 32-bit transactions and combines them into a single 64-bit AHB-Lite transaction whenever the merging conditions are satisfied. A frequency-aware FIFO mechanism is incorporated to support communication between high-speed and low-speed clock domains without introducing unnecessary pipeline stalls.

To ensure functional correctness, the complete RTL design is verified using SystemVerilog testbenches, directed and constrained-random test cases, scoreboards, functional coverage, and SystemVerilog Assertions (SVA). The proposed bridge is compared with a conventional non-merging baseline architecture in terms of communication efficiency, bandwidth utilization, and clock cycles. The design demonstrates how request merging and efficient buffering can significantly improve bus utilization while preserving protocol correctness.

***Index Terms*—System-on-Chip, ICB, AHB-Lite, Bus Bridge, Request Merging, RTL Design, SystemVerilog, FIFO, Assertion-Based Verification, RISC-V**

I. Introduction

The increasing complexity of modern System-on-Chip (SoC) designs has created a growing demand for efficient communication between different hardware components. Processors, memories, peripherals, accelerators, and communication interfaces continuously exchange data through on-chip buses. Since these components often operate at different clock frequencies and use different communication protocols, bus bridges have become an essential part of almost every SoC architecture.

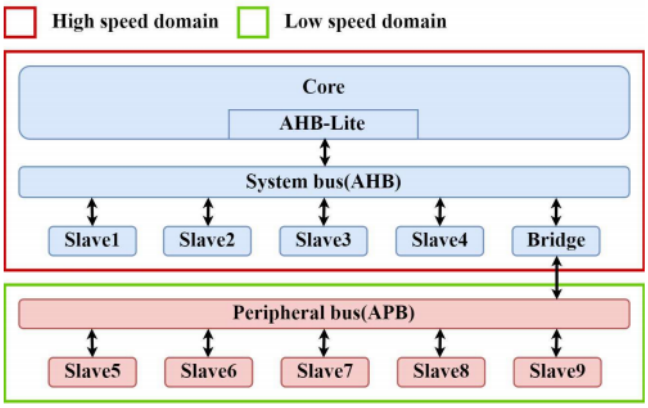


Fig. 1: Simplified block diagram of an SoC, using two different buses to manage communication.

As semiconductor technology advances, designers are integrating larger numbers of processing elements into a single chip. Although computation capability continues to improve, communication between modules frequently becomes the limiting factor in overall system performance. Inefficient bus communication increases latency, reduces throughput, wastes bandwidth, and may lead to unnecessary power consumption. Consequently, designing efficient bus bridges has become an important research area in digital system design.

Many industrial SoCs employ the ARM Advanced Microcontroller Bus Architecture (AMBA) because of its modularity, scalability, and widespread industry support. Among the AMBA family, the Advanced High-performance Bus Lite (AHB-Lite) is commonly used as the system bus for connecting processors, memories, and high-performance peripherals. Inside processor cores, however, several lightweight proprietary or semi-standard buses are often used for internal communication. One such protocol is the Internal Chip Bus (ICB), which provides separate request and response channels together with a simple valid-ready handshake mechanism.

In practical SoC implementations, the processor-side bus may operate using a 32-bit data width at a relatively high clock frequency, while the system-side AHB-Lite bus may

operate with a 64-bit data width at a lower frequency. A conventional bridge, CMSDK, converts each 32-bit transaction independently into a corresponding AHB-Lite transfer. Although this approach is functionally correct, it cannot fully utilize the available bandwidth of the 64-bit system bus. Paper [1] identified two concrete deficiencies in a CMSDK-based bridge connecting a 32-bit proprietary CBus to a 64-bit AHB-Lite bus: (1) each 32-bit transfer occupies only half of the 64-bit AHB-Lite data path, and (2) the AHB-to-AHB synchronization-down bridge used for frequency conversion breaks AHB Lite's pipelined address/data overlap, introducing a stall between every pair of transactions. Their proposed MergFS architecture addresses both issuses.

Recent research has shown that communication efficiency can be improved by intelligently combining multiple transactions before forwarding them to the wider bus. The recently proposed MergFS architecture demonstrated that merging two adjacent 32-bit CBus transactions into a single 64-bit AHB-Lite transaction significantly improves bandwidth utilization while reducing the number of required clock cycles. In addition, MergFS introduced a frequency-aware buffering mechanism that enables efficient communication between clock domains operating at different frequencies.

Although the original MergFS architecture demonstrates promising performance improvements, it is specifically designed for the proprietary CBus protocol. Many modern open-source RISC-V platforms instead employ the Internal Chip Bus (ICB), whose interface characteristics differ from those of CBus. Consequently, the original request-merging logic and protocol conversion mechanism cannot be directly applied to ICB-based systems. This creates an opportunity to extend the request-merging concept toward a more reusable and widely applicable bridge architecture.

This paper proposes ICB-MergFS, a request-merging ICB-to-AHB-Lite bridge that adapts the fundamental concepts of MergFS to the ICB protocol. The proposed design preserves the independent request and response channels of ICB while introducing conservative request-merging logic, frequency-aware buffering, and protocol conversion to AHB-Lite. The bridge is designed using synthesizable SystemVerilog RTL and verified through class based testbench and assertion-based verification techniques.

Compared with a conventional non-merging bridge, the proposed architecture aims to improve bandwidth utilization, reduce communication latency, and increase overall bus efficiency while maintaining functional correctness and protocol compliance.

II. Problem Statement

How can two adjacent 32-bit ICB transactions be safely merged into one 64-bit AHB-Lite transaction while preserving correctness, response ordering, protocol handshaking, and timing behavior across different clock-frequency ratios, and improving overall communication efficiency?

To answer this question, two bridge architectures are implemented and compared. The first is a conventional baseline bridge that forwards every request independently. The second is the proposed ICB-MergFS bridge that performs request merging whenever the predefined conditions are satisfied, and uses frequency-aware buffering to improve transfer efficiency.

Table I: ICB protocol signals (controller-side direction shown)

Signal	Dir.	Description
`icb_cmd_valid`	output	Master has a request ready
`icb_cmd_ready`	input	Slave can accept the request
`icb_cmd_addr`	output	Byte address of the request
`icb_cmd_read`	output	0 = write, 1 = read
`icb_cmd_wdata`	output	Write data
`icb_cmd_size`	output	00 = byte, 01 = half, 10 = word
`icb_rsp_valid`	input	Slave has a response ready
`icb_rsp_ready`	output	Master can accept the response
`icb_rsp_rdata`	input	Read data
`icb_rsp_err`	input	1 = error occurred

III. Background

This section briefly introduces the communication protocols used in this work and explains the need for protocol conversion between them.

A. Internal Chip Bus (ICB)

The Internal Chip Bus (ICB) is a lightweight on-chip communication protocol that is widely used in open-source RISC-V based systems. The protocol provides independent command channel(`icb_cmd_*`) on which the core issues requests, and a response channels(`icb_rsp_*`) on which the bridge or peripheral returns results. Communication follows a valid-ready handshake mechanism that enables both the sender and receiver to control data transfer.

Unlike more complex protocols such as AXI, ICB is designed to provide a simple interface with lower hardware overhead while maintaining good communication efficiency. Because of its simplicity, ICB is well suited for processor subsystems, tightly coupled memories, and lightweight peripherals.

B. AHB-Lite Bus

The Advanced High-performance Bus Lite (AHB-Lite) is one of the communication standards defined in the ARM AMBA specification. It is widely adopted in embedded processors because of its simple architecture, pipelined communication mechanism, and support for high-performance memory interfaces.

An AHB-Lite transaction normally consists of two phases:

1) Address Phase
2) Data Phase

Unlike ICB, AHB-Lite uses a pipelined transfer mechanism where the address phase of current transaction can overlap with the data phase of the last transaction. This overlapping improves throughput when multiple transactions are executed consecutively.

Table II: AHB-Lite protocol signals (master-side direction shown)

Signal	Dir.	Description
HCLK	input	Bus clock (rising edge active)
HRESETn	input	Active-low asynchronous reset
HADDR[31:0]	output	Byte address of the transfer
HTRANS[1:0]	output	00=IDLE, 01=BUSY, 10=NONSEQ, 11=SEQ
HWRITE	output	0 = read, 1 = write
HSIZE[2:0]	output	000=byte, 001=half, 010=word, 011=dword
HWDATA[31:0]	output	Write data (master → slave)
HRDATA[31:0]	input	Read data (slave → master)
HREADY	input	1 = transfer complete; 0 = wait state
HRESP	input	0 = OKAY, 1 = ERROR

C. Need for an ICB-to-AHB-Lite Bridge

Modern SoCs frequently connect processor cores operating on an internal communication protocol with memories and peripherals connected through AHB-Lite. Since these buses differ in protocol, timing behavior, and data width, direct communication between them is not possible.

An interface bridge performs several important tasks:

- Protocol conversion
- Data width conversion
- Address translation
- Response generation
- Clock-domain synchronization
- Buffer management

D. Clock-Enable Slow Domain

Instead of a second clock, the slow domain is represented by an enable $clk2en$ that is high for one of every freq_ratio cycles of a single clock $clk1$. Slow-side state changes only on edges where $clk2en$ = 1. This avoids synchronisers and asynchronous FIFOs, although all slow-side handshakes and pointers must be qualified by $clk2en$.

IV. Baseline ICB-to-AHB-Lite Bridge

Before developing the proposed request-merging bridge, a conventional baseline bridge is implemented. The baseline architecture serves as the reference design for evaluating the improvements achieved by the proposed approach.

The baseline bridge performs protocol conversion without modifying the incoming transaction sequence. Every valid ICB request is converted into one corresponding AHB-Lite transaction, irrespective of neighbouring requests.

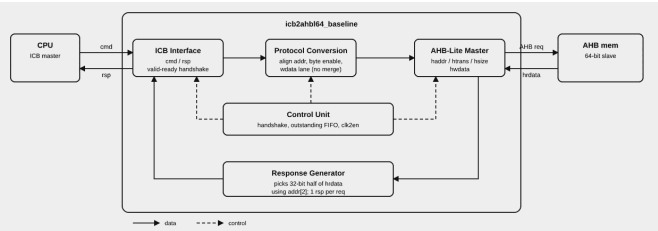


Fig. 2: Baseline ICB-to-AHB-Lite bridge architecture.

The baseline bridge correctly converts read and write requests while preserving transaction ordering. However, each 32-bit request occupies an entire 64-bit AHB-Lite transfer. Consequently, only half of the available data bandwidth is utilized for every transaction. Furthermore, the bridge does not attempt to reduce the number of AHB-Lite transactions, resulting in increased communication latency under sequential workloads.

V. Proposed ICB-MergFS Architecture

The proposed architecture extends the baseline bridge by introducing request merging and frequency-aware buffering. Three bllock: the ICB Merge Unit, which implements request merging and response splitting; the Frequency-Aware FIFO Unit, which buffers requests and responses across the clock rate boundary while preserving AHB-Lite pipelining; and ICB2AHBL64, which performs the actual ICB to-AHB-Lite protocol conversion in the slow clock domain. Instead of immediately forwarding every incoming request, the bridge temporarily stores requests and checks whether adjacent transactions can be safely merged.

Whenever two neighbouring requests satisfy the predefined merging conditions, they are combined into a single 64-bit AHB-Lite transfer. Otherwise, the requests are forwarded individually.

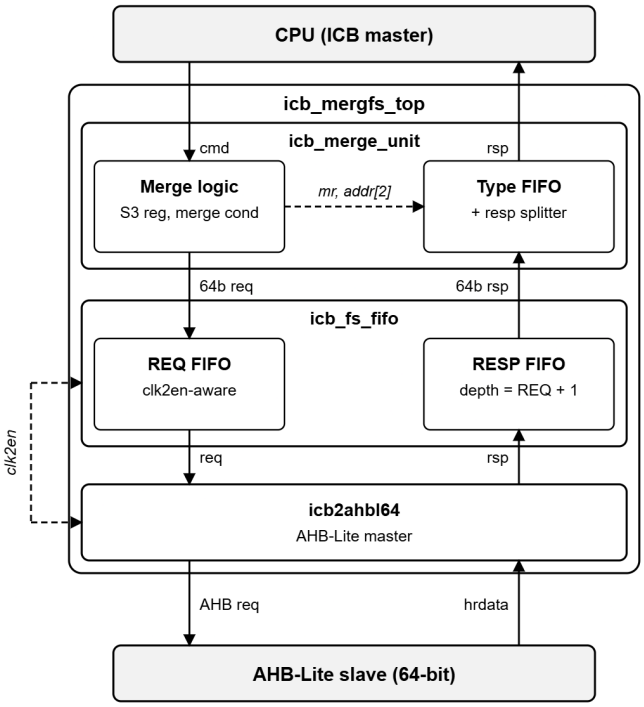


Fig. 3: Overall architecture of the proposed ICB-MergFS bridge.

A. ICB Merge Unit

The merge unit converts 32-bit ICB requests into 64-bit requests and splits 64-bit responses back into one or two 32-bit responses, following the same two-stage organization as

CBusMerg in the original MergFS design [1], restated here in terms of ICB signals.This reduces the total number of bus accesses, improves utilization of the 64-bit data bus, and decreases communication latency.

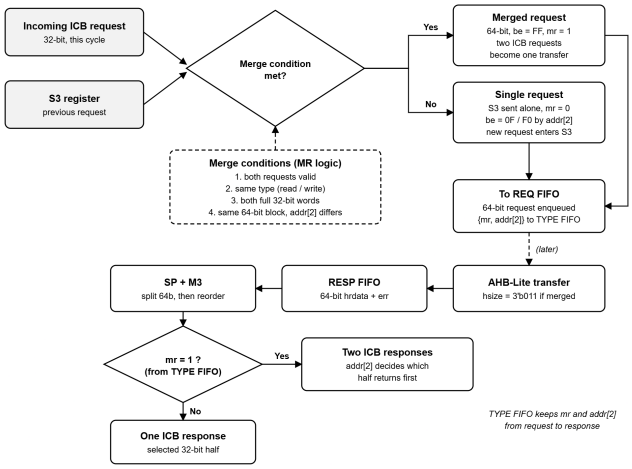


Fig. 4: Flowchart showing the request-merging process.

Request merging logic. A pipeline register, *S3*, captures and temporarily stores, one pending 32-bit request using a standard valid/ready handshake. A combinational module, *MR*, compares the request held in S3 against the request currently presented on the ICB command channel and asserts a one-bit `merged` flag when all of the following conditions hold:

1) Both requests are valid.
2) `icb_cmd_read` matches between the two requests (both reads or both writes).
3) `icb_cmd_size` equals `2'b10` (word) for both requests.
4) Both addresses belong to the same aligned 64-bit address block i.e., `addr[31:3]` is identical between the two requests
5) `addr[2]` differs between the two requests; one addresses the low 32-bit half, the other the high half.

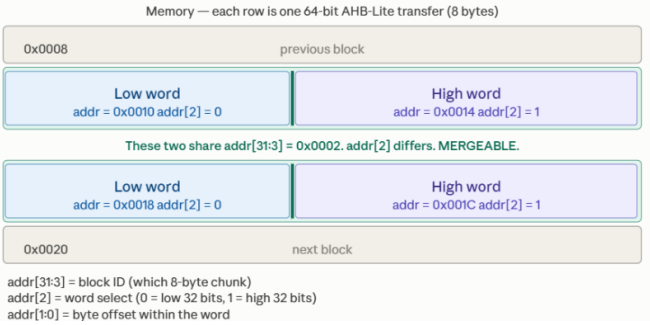


Fig. 5: A 64-bit AHB-Lite transfer spans 8 bytes, so addr[31:3] identifies the aligned block and addr[2] selects the 32-bit half within it.

Response splitting logic. A combinational splitter, *SP*, decomposes an incoming 64-bit response into `resp1 = rdata[31:0]` and `resp2 = rdata[63:32]`. A multiplexer, *M3*, uses the saved metadata to decide delivery order: if the earlier of the two original requests had `addr[2]`=0, `resp1` is returned first and `resp2` second; if `addr[2]`=1, the order is reversed. For an unmerged entry only the corresponding single half is returned.

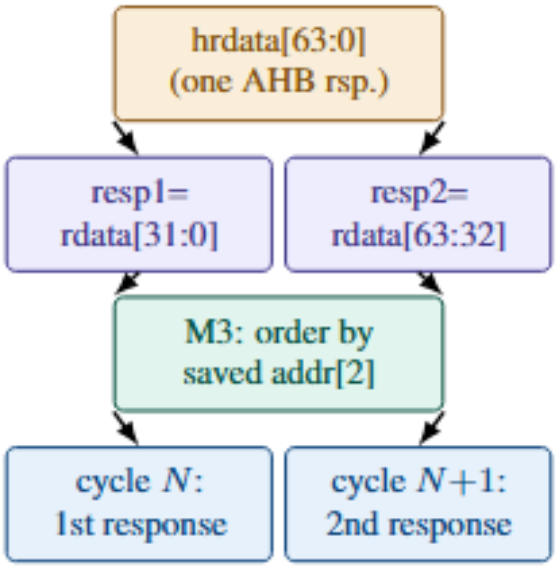


Fig. 6: Response-splitting. The metadata bit saved at merge time determines delivery order; the RESP FIFO entry is dequeued only after the second response is accepted.

B. Frequency-Switching FIFO Buffering Unit

The bridge connects an intra-core ICB operating on the fast clock `clk1` to a 64-bit AHB-Lite bus operating on the slower clock `clk2en`, where `clk1`/`clk2` is an integer ratio N that may change at run time. Following MergFS [1], clk2 is not generated as a separate clock. A clock-enable signal `clk2en` is asserted for one `clk1` cycle out of every N, and all slow-domain state is updated only on `clk1` edges where `clk2en` $= 1$. Every register in the design is therefore clocked by `clk1`, and no clock-domain-crossing synchronizers are required.

This unit buffers requests and responses across the `clk1`/`clk2` boundary, following the three-FIFO organization of CBusFS from base paper [1]: REQ_FIFO (clock-enable-gated, depth 2), REQ_TYPE_FIFO (standard synchronous, depth 8), and RESP_FIFO (standard synchronous, depth 3, one entry deeper than REQ_FIFO since AHB-Lite provides no mechanism for a controller to reject a peripheral response). Together they fulfill three functions: (i) buffering merged 64-bit requests while moving them from the fast to the slow side, (ii) recording per-request information needed to split responses, and (iii) buffering 64-bit responses so that the fast side has enough cycles to split them into 32-bit ICB responses.

Table III: FIFOs of the proposed bridge: placement, clock behaviour and payload.

FIFO	Module	Write side	Read side	Width × depth	Payload
REQ FIFO	`icb_fs_fifo`	`clk1`, every cycle (fast)	`clk1` gated by `clk2en` (slow)	106×2	`{mr, wdata, be, we, addr}`
TYPE FIFO	`icb_merge_unit`	`clk1` (fast)	`clk1` (fast)	2×8	`{mr, addr[2]}`
RESP FIFO	`icb_fs_fifo`	`clk2` cycles only (slow)	`clk1`, every cycle (fast)	65×3	`{err, rdata}`

1) FIFO Operation: The FIFO consists of

- Write Pointer
- Read Pointer
- FIFO Memory
- Full flag
- Empty flag
- Clock Enable Logic

Incoming requests are written into the FIFO using the processor-side clock. Transactions are removed using the AHB-Lite clock whenever the destination bus becomes ready. The design supports different clock-frequency ratios:

- 1:1
- 2:1
- 3:1
- 4:1
- 5:1

These configurations allow the bridge to evaluate communication efficiency under different operating conditions.

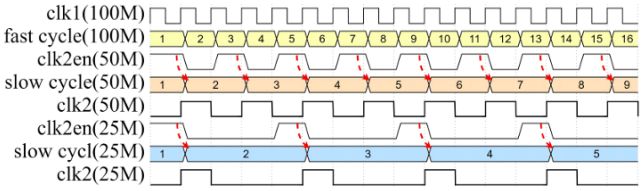


Fig. 7: Clock division operation for 2:1 and 4:1 clock divide ratios.

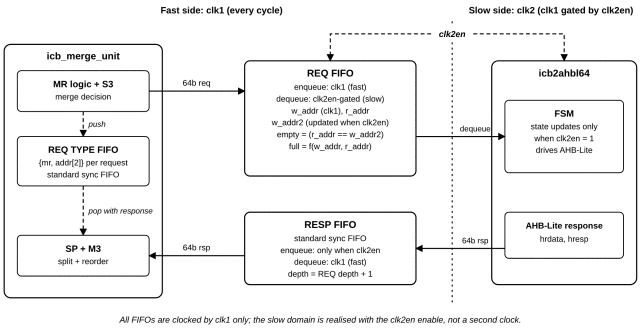


Fig. 8: Frequency-aware FIFO architecture.

2) REQ FIFO: A standard synchronous FIFO derives its empty flag by comparing the read pointer with the write pointer. In a clock-ratio system this exposes a newly written entry to the slow-side reader immediately, although it is stable for only one `clk1` cycle, which is shorter than one `clk2` period when $N \geq 2$. Registers on the slow side could then sample unstable data and violate setup time.

REQ_FIFO resolves this by maintaining two write-pointer copies. *WP_logic* updates `w_addr` on every `clk1` edge and drives the `full` flag observed by the fast domain. *WP_logic2* updates a second copy, `w_addr2`, only on `clk2en` edges where `clk2en`=1, i.e., at most once per `clk2` period and the `empty` flag observed by the slow domain is derived from `w_addr2` rather than `w_addr` so an entry becomes visible to the consumer only at a `clk2` boundary, while the producer still sees a free slot as soon as the entry is read. Because `clk2en` is asserted at a fixed, known phase relationship to `clk2`, this guarantees that `empty` can only transition in alignment with the slow clock, eliminating the setup-time hazard without resorting to a general-purpose asynchronous CDC synchronizer.

$$w_addr2 \leftarrow \begin{cases} w_addr, & \texttt{clk2en} = 1, \\ w_addr2, & \text{otherwise.} \end{cases} \tag{1}$$

3) TYPE FIFO: For every 64-bit request that is accepted downstream, the TYPE FIFO stores two bits: the merge flag `mr` and the alignment bit `addr[2]` of the first request. Entries are pushed at the request handshake and popped when the last response beat of that request has been returned to the ICB. Its outputs control the response splitter and multiplexer: `mr` decides whether one or two 32-bit responses are issued, and `addr[2]` decides which half of the 64-bit response is returned first. The TYPE FIFO also qualifies `icb_rsp_valid_o`, so that a response is never forwarded without a recorded request, and its pop signal generates the dequeue request for the RESP FIFO. It operates entirely on the fast side. Unlike the REQ and RESP FIFOs it does not back-pressure the request path, so its depth must cover the number of outstanding transactions.

4) RESP FIFO: The RESP FIFO receives the 64-bit response (`rdata`, `err`) from the AHB-Lite master. The master asserts `resp_valid` only when `clk2en` $= 1$, and the FIFO must accept it in that cycle; `resp_ready_o` is therefore required to be high whenever a response is presented. The fast side dequeues entries on every `clk1` cycle through `resp_ready_i`, which is driven by the response splitter. Buffering the response gives the splitter the two or more `clk1` cycles it needs to return a merged response as two 32-bit ICB responses, even though the slow side produced it in a single

clk2 cycle. To avoid loss of responses during frequency switching, the RESP FIFO is one entry deeper than the REQ FIFO; our implementation uses depths of 2 and 3, respectively.

5) Run-time frequency switching: Because the pointers and memories of all three FIFOs are clocked by clk1, changing N only changes how often clk2en is asserted. No FIFO needs to be flushed or reconfigured, and the FIFO depths above remain sufficient for every ratio.

VI. Protocol Conversion Unit

After request merging, transactions enter the Protocol Conversion Unit. This block serves as the protocol conversion interface between the ICB-based request-processing subsystem and a 64-bit AHB-Lite slave. Its primary function is to translate incoming ICB requests into AHB-Lite transactions while coordinating address generation, write-data transfer, and response handling. The module accepts buffered requests containing the address, read/write control, byte-enable information, write data, and merge status. Based on these attributes, it determines the appropriate AHB-Lite transfer address and size.

The conversion unit comprises request-processing logic, a request register, address/data selection logic, transfer-control logic, an AHB-Lite master interface, and response-processing logic. The request-processing stage interprets the incoming request and determines the address offset and transfer size. For an unmerged 32-bit access, the upper or lower 32-bit word is selected according to the byte-enable information. For a merged 64-bit access, the original aligned block address is retained. The transfer size is configured using the AHB-Lite HSIZE signal, while HWRITE specifies the transfer direction. It performs four main functions:

1. Request processing: Accepts buffered requests containing the address, read/write control, byte enables, and write data.

2. Address and size conversion: Selects the correct address and transfer size. For an unmerged 32-bit access, it selects the lower or upper 32-bit word; for a merged 64-bit access, it uses the full 64-bit transfer size.

3. Transfer control and data handling: Uses registered control signals and status flags to coordinate request launches, AHB-Lite readiness, and write-data timing.

4. Response handling: Forwards the AHB-Lite read data (HRDATA) and response status (HRESP) to the upstream response interface.

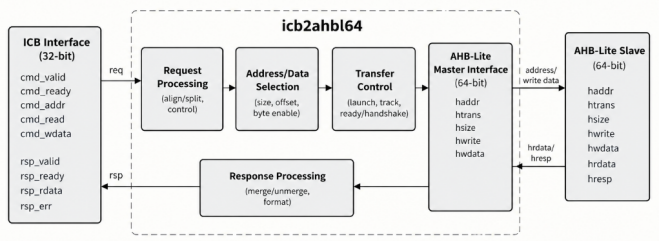


Fig. 9: ICB-to-AHB-Lite protocol conversion block.

VII. RTL Implementation

The proposed ICB-MergFS bridge is implemented using synthesizable SystemVerilog RTL. The design follows a modular architecture to simplify implementation, debugging, verification, and future extensions. Each functional block is developed independently before integrating the complete bridge.

The RTL implementation consists of the following major modules:

- ICB Interface
- ICB Merge Unit
- Frequency-Aware FIFO Unit
- Protocol Conversion Unit (icb2ahbl)
- Top-Level Controller
- Testbench module

Fig.10 shows the organization of the RTL modules.

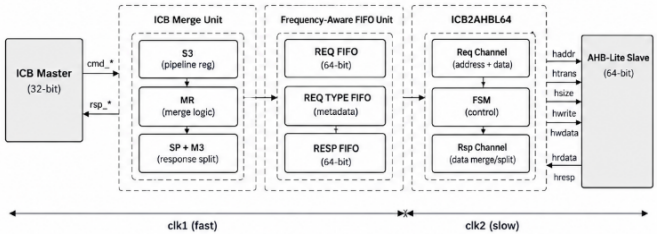


Fig. 10: RTL architecture of the proposed ICB-MergFS bridge.

VIII. Verification Methodology

Verification is an important phase of digital hardware design because functional correctness must be ensured before synthesis and implementation. The proposed bridge is verified using a combination of simulation-based verification and assertion-based verification.

The verification environment is developed entirely in SystemVerilog. Class-based verification methodology. The complete verification environment consists of:

- Transaction
- Generator class
- Driver class
- Monitor class
- Scoreboard class
- Golden Memory Model
- Functional Coverage
- SystemVerilog Assertions

The transaction generator produces different traffic patterns which are applied to the DUT (Design Under Test). The monitor observes all bus transactions and forwards them to the scoreboard. The scoreboard compares the DUT output with the expected output generated by the reference memory model.

A. Directed Test Cases

Initially, directed test cases are developed to verify individual functionalities of the bridge.

The following scenarios are considered:

1) Single 32-bit read transaction.
2) Single 32-bit write transaction.

3) Two adjacent mergeable read transactions.
4) Two adjacent mergeable write transactions.
5) Non-mergeable read-write pair.
6) Non-mergeable write-read pair.
7) Non-mergeable address block unaligned pair.
8) Back-to-back mergeable transactions.
9) FIFO full condition.
10) FIFO empty condition.
11) Reset during idle condition.
12) Different frequency ratios.

These tests verify the correctness of every major functional block before random testing is performed.

B. Constrained-Random Verification

After successful execution of the directed tests, constrained-random verification is performed.

Random transaction sequences are generated with variations in:

- Address
- Read/Write command
- Clock-frequency ratio
- Valid-ready stalls
- Mergeable transaction patterns
- Non-mergeable transaction patterns

The random tests improve design confidence by exploring a much larger input space than directed testing alone.

C. SystemVerilog Assertions

SystemVerilog Assertions (SVA) are incorporated to continuously monitor protocol correctness during simulation.

The developed assertions verify:

1) No request is lost.
2) No response is duplicated.
3) Response order is preserved.
4) Merged read response is split correctly.
5) Merged write data is packed correctly.
6) FIFO does not overflow.
7) FIFO does not underflow.
8) AHB-Lite address is aligned correctly for 64-bit merged transfers.
9) AHB-Lite transfer size is correct.
10) Valid/ready handshake is obeyed.
11) Reset brings the bridge to a known safe state.

Assertion-based verification enables protocol violations to be detected immediately during simulation.

IX. Results

A. Baseline Outputs

```
======================================================================
[SCOREBOARD REPORT] checked=312  passed=312  failed=0  (reads=213 writes=99)
[SCOREBOARD] ALL CHECKS PASSED
======================================================================
======================================================================
[COVERAGE REPORT] cg_txn = 100.00%   cg_ahb = 66.67%
[COVERAGE REPORT] observed merge ratio = 0.000  (merged_xfers=0 unmerged_xfers=312)
======================================================================
***** TEST PASSED  (checked=312) *****
Simulation complete via $finish(1) at time 11575 NS + 4
```

Fig. 11: Baseline (without merging) simulation log output



Fig. 12: Baseline (without merging) simulation output waveform

B. Proposed ICB-MergFs Outputs

```
======================================================================
[SCOREBOARD REPORT] checked=312  passed=312  failed=0  (reads=213 writes=99)
[SCOREBOARD] ALL CHECKS PASSED
======================================================================
======================================================================
[COVERAGE REPORT] cg_txn = 100.00%   cg_ahb = 86.67%
[COVERAGE REPORT] observed merge ratio = 0.872  (merged_xfers=136 unmerged_xfers=40)
======================================================================
***** TEST PASSED  (checked=312) *****
Simulation complete via $finish(1) at time 8325 NS + 4
```

Fig. 13: Baseline (without merging) simulation log output

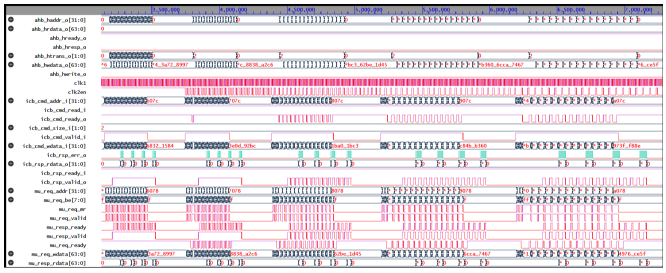


Fig. 14: Baseline (without merging) simulation output waveform

X. Performance Evaluation

The proposed bridge is evaluated using multiple workloads with different transaction characteristics.

Workload	Description	Expected Merge Ratio
W1	Alternating read and write transactions	Low
W2	Sequential 32-bit read transactions	High
W3	Sequential 32-bit write transactions	High
W4	Random mixed read/write transactions	Medium/Variable
W5	Non-mergeable address pattern	Low/Zero

Fig. 15: Evaluation Workloads

The following metrics are used.

- Total clock cycles
- Average cycles per transaction
- Throughput
- Merge ratio
- Bandwidth utilization

Table V provides a placeholder for the final comparison. The throughput improvement is approximately 26.01%

Workload	N	Baseline	Merging
W2 seq read	1–5	50%	100%
W3 seq write	1, 2	50%	100%
W1 alt R/W	1	50%	50%
W5 non-mergeable	1	50%	50%
W4 random mixed	1	50%	50%

Fig. 16: Bandwidth Utilization per Workload

Useful data transferred	$136 \times 64 + 40 \times 32 = 9984$ bits	$312 \times 32 = 9984$ bits
Maximum bus capacity over those transfers	$176 \times 64 = 11264$ bits	$312 \times 64 = 19968$ bits
Aggregate bus-width utilization	**88.64%**	**50.00%**

Fig. 17: Average Bandwidth Utilization

In Table IV, For N 2, the baseline costs about N cycles per transaction. Merging costs about max(N/2, 1), which is the trend in the reference paper [1] (CBus-MergFS) Table XI. The cycle reduction is about 50% for merge-friendly workloads. The paper's 75% is measured against a non-pipelined CMSDK baseline which is not applicable here.

Table IV: Cycles and cycles per transaction for different workloads.

			Baseline		Merging		
Workload	***N***	**Txns**	**Cycles**	**C/Txn**	**Cycles**	**C/Txn**	**Red. (%)**
W2 Seq. read	1	16	23	1.44	16	1.00	30.4
W2 Seq. read	1	16	23	1.44	16	1.00	30.4
W3 Seq. write	1	16	23	1.44	16	1.00	30.4
W1 Alt. R/W	1	16	23	1.44	23	1.44	0.0
W5 Non-merge	1	8	11	1.38	11	1.38	0.0
W4 Random	1	16	50	3.12	50	3.12	0.0
W2 Seq. read	1	32	47	1.47	32	1.00	31.9
W2 Seq. read	2	32	63	1.97	32	1.00	49.2
W2 Seq. read	3	32	94	2.94	47	1.47	50.0
W2 Seq. read	4	32	125	3.91	62	1.94	50.4
W2 Seq. read	5	32	156	4.88	77	2.41	50.6
W3 Seq. write	2	64	127	1.98	64	1.00	49.6

XI. Discussion

The proposed ICB-MergFS bridge is expected to improve communication efficiency by reducing the number of AHB-Lite transactions whenever adjacent requests satisfy the merging conditions. Better utilization of the 64-bit bus can reduce communication latency and improve throughput, particularly for sequential memory-access patterns.

In the baseline design, although no merging request is present, the pipelining feature of the AHB protocol is already in use i.e., the address phase of the current transaction begins during the data phase of the previous one. Since this pipelining is already accounted for in the baseline, the claimed 75% reduction in clock cycles for the proposed design does not hold.

Table V: Performance Comparison

Metric	Baseline	Proposed
Total simulation time	10,495 ns	8,325 ns
Total transactions	312	312
Merged txns	0	136
Unmerged txns	312	40
Merge ratio	0%	87.18%
Total clk1 cycles	1049.5	832.5
Average Cycles/txn	3.36	2.67
Throughput (M txns/sec)	29.73	37.48
Reduction in transfers	0%	43.6%
Area	XX	XX
Power	XX	XX
Maximum Frequency	XX	XX

The introduction of a frequency-aware FIFO also helps reduce stalls when the processor and memory subsystems operate at different clock frequencies. Although the proposed design introduces additional logic for request merging and buffering, the increase in hardware resources is expected to be relatively small compared with the improvement in communication efficiency.

Baseline: 765 cycles in total.
Merging: 446 cycles in total.
Cycles saved: 319.
Overall Clock cycle reduction 41.7%

XII. Future Work

Several improvements can be explored in future work.

- Support for partial-byte transfers.
- Handling of unaligned memory accesses.
- Multi-master bus support i.e., AHB protocol.
- AXI protocol conversion.
- UVM-based verification.
- Formal verification.
- FPGA implementation.

XIII. Conclusion

This paper presented ICB-MergFS, instead of directly reproducing the original CBus-to-AHB-Lite bridge, the project extends the idea toward ICB-to-AHB-Lite conversion for RISC-V based System-on-Chip designs, while preserving protocol correctness and response ordering.

The bridge combines eligible adjacent 32-bit transactions into a single 64-bit AHB-Lite transfer, thereby improving bandwidth utilization and reducing communication latency. A frequency-aware FIFO enables efficient communication between clock domains with different operating frequencies. Functional correctness is ensured through SystemVerilog-based verification using directed tests, constrained-random testing, scoreboards, and SystemVerilog Assertions.

The work is technically meaningful because it addresses bus-width conversion, request merging, frequency-ratio handling, and assertion-based verification. The proposed work provides a practical architecture for improving bus communication efficiency and can serve as a foundation for future research on high-performance SoC interconnects.

REFERENCES

[1] Z. Cao, L. Xu, Z. Peng, Y. Dong, H. Ruan, C. Zeng, H. Zhao, and J. Luo, "MergFS: Efficient Bridging of a 32-bit High-Speed Intra-Core Bus to a 64-bit Low-Speed AHB-Lite Bus," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 73, no. 1, pp. 182–195, Jan. 2026, doi: 10.1109/TCSI.2025.3597140.

[2] Nuclei System Technology, "Hummingbirdv2 E203 Core and SoC," [Online]. Available: https://doc.nucleisys.com/hbirdv2/core/core.html

[3] Arm Ltd., "AMBA AHB Protocol Specification," IHI 0033C, 2021. [Online]. Available: https://developer.arm.com/documentation/ihi0033/c/

[4] Arm Ltd., "Arm Cortex-M System Design Kit," DDI 0479D, Revision R1p1, 2017. [Online]. Available: https://developer.arm.com/documentation/ddi0479/d